Project Report : Bee World Simulation

Name: Abu Fatah Mohammed Faisal

Student Number: 23107667

Date: 17 May, 2025

Overview

The Bee World Simulation is an agent-based Python program designed to model and visualize the collective foraging behaviour of bees within a customizable 2D environment. With its own state machine, each bee in this simulation functions as an independent agent that alternates between states including resting, searching, gathering, returning, and communicating in response to both internal logic and outside stimuli. The environment consists of a grid populated with flowers of varying nectar levels, obstacles that restrict movement, and a central hive which serves as the base for nectar storage and information sharing.

Users can configure the simulation parameters, including world size, number of bees, flowers, obstacles, simulation steps, and the communication strategy used by the bees (random or directed). The simulation can be run in interactive mode, where users input parameters directly, or in batch mode using CSV files for terrain and configuration. Real-time visualization is provided via matplotlib, displaying both the main world (with bee positions, flowers, obstacles, and hive) and the hive's interior, giving insight into bee activity and resource accumulation. The simulation outputs detailed step-by-step summaries to the terminal, supporting both qualitative observation and quantitative analysis of bee behaviours and environmental changes.

User Guide

Setup & Requirements

- Python 3.8+ is required.
- Install dependencies: *pip install numpy matplotlib*

- Place all code files in the correct directories:
    - models/bee.py, models/environment.py, models/hive.py, models/flower.py, models/terrain.py
    - utils/config_loader.py, utils/visuals.py
    - beeworld.py (main script)

Running the Simulation

Interactive mode: *python3 beeworld.py -i*

You will be prompted for world size, number of bees, flowers, obstacles, steps, and communication strategy.

Batch mode (with CSV files): *python3 beeworld.py -f map1.csv -p params1.csv*

- map1.csv: Terrain map (obstacles, etc.)
- params1.csv: Simulation parameters

Output

- Terminal: Step-by-step summary of hive nectar, bee missions, flower nectar levels, etc.
- Visualization: Animated matplotlib window showing bee positions, flower states, obstacles, and hive interior.

Traceability Matrix

The following traceability matrix links each implemented feature to its corresponding code, testing method, completion status, and date:

Feature	Code Reference	Test Reference	Status	Date Completed
1. Bee movement and state machine	WorkerBee class in bee.py	Visual check, print statements	P	05/05/2025
2. Flower placement and nectar logic	Blossom class in flower.py	Visual, step output	P	07/05/2025
3. Hive storage and knowledge	Hive class in hive.py	Visual, print output	P	09/05/2025
4. Environment setup	Environment class in environment.py	Visual, print output	P	10/05/2025
5. Bee communication	share_flower_info in bee.py	Print output, scenario runs	P	12/05/2025
6. Obstacle placement	Obstacle class in terrain.py	Visual, print output	P	13/05/2025
7. Visualization	visuals.py	Visual check	P	15/05/2025
8. Config loading	config_loader.py	Batch run with CSVs	P	16/05/2025
9. Parameter sweep	N/A	N/A	S	N/A

Discussion

Implemented Features

Bee Agent Logic: With a finite state machine, each WorkerBee is represented as an autonomous agent that alternates between various states, including resting, hive, searching, gathering, returning, and holding. The foraging cycles seen in honeybee colonies in the real world are modelled by this state-driven behaviour. Agents make choices about when to leave the hive, which flowers to target, and when to return by interacting dynamically with the environment and the hive. Similar to research on swarm intelligence and collective decision-making, the simulation can investigate both decentralized and coordinated foraging behaviours because the sharing of flower location information is controlled by the chosen communication strategy (random or directed).

Environment: The Environment class serves as the main controller for the simulation, controlling the 2D grid world and arranging the hive, flowers, obstacles, and bees. In addition to tracking global variables like simulation time and terrain configuration, it is in charge of starting the simulation and updating agent states at each timestep. The environment offers a realistic and adaptable setting for agent-based modelling by guaranteeing that interactions between agents and objects (such as bees running into obstacles or flowers) are handled consistently.

Hive: Keeping track of the amount of nectar stored, the hive's maximum capacity, and a list of flower locations that the colony is aware of, the Hive class serves as the primary resource centre for the colony. It also regulates signals, which might be extended to enable more complex communication or recruiting procedures. The function of the hive reflects the social and

organizational structure of real bee colonies, where gathering resources and exchanging information are critical to survival.

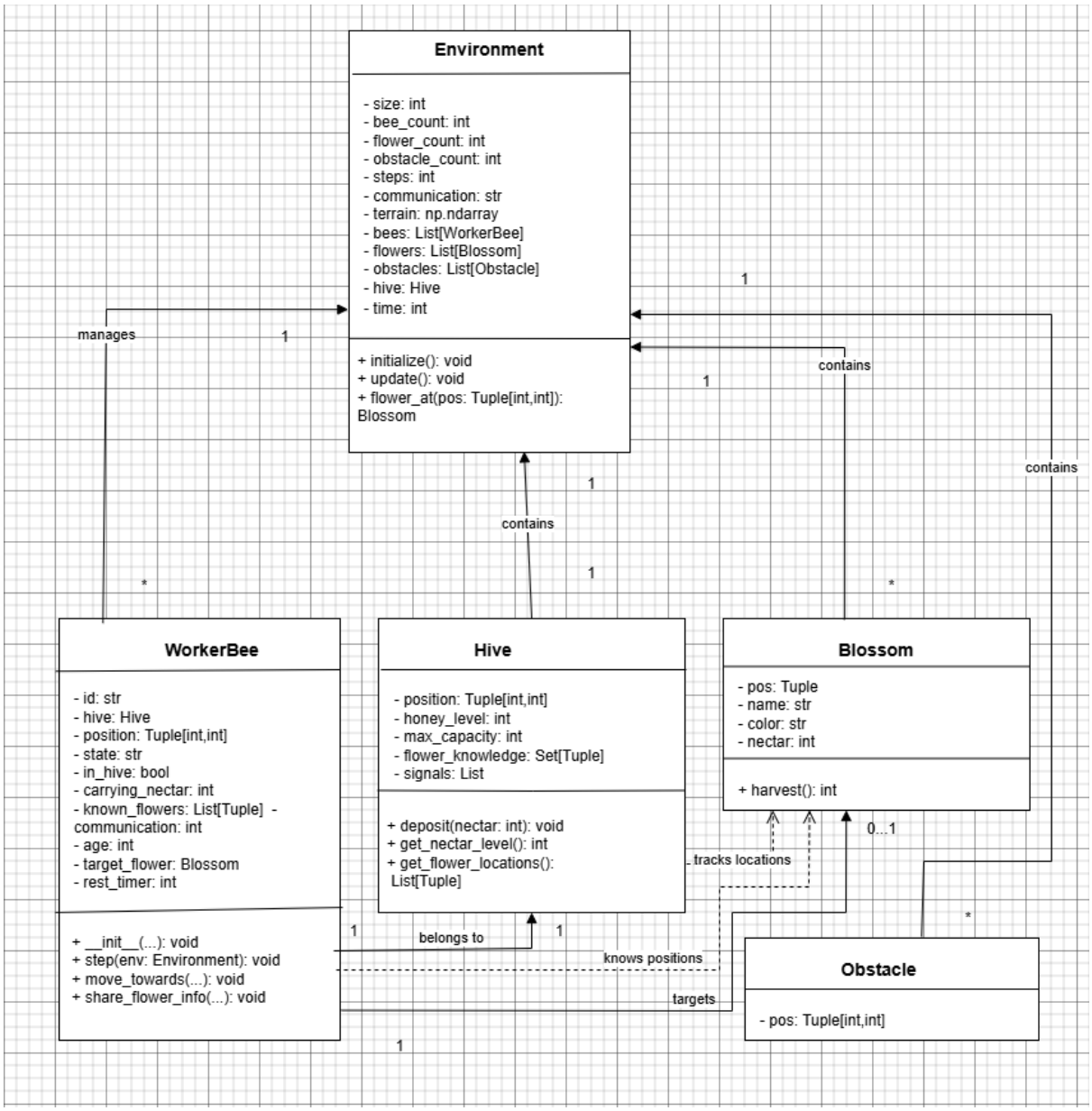
Flowers: Each flower patch in the environment is represented by a blossom object, which has a unique name, location, colour, and nectar content. The condition of the flowers is constantly updated as nectar is collected, and visiting bees may exhaust them. Research on pollinator eating and ecological dynamics replicates the temporal and geographical diversity of floral supplies.

Obstacles: Barriers or impassable terrain within the world grid are examples of obstacle objects, which have a direct impact on bee mobility and foraging effectiveness. As observed in ecological and robotics research, the simulation can investigate how environmental complexity affects agent navigation, resource acquisition, and overall colony performance by adding obstacles.

Visualization: Matplotlib is used in the simulation to provide dual-panel visualization in real time. The world grid, bee positions (color-coded by state), flowers (by nectar level), obstacles, and the hive are all displayed in the main subplot. The number of bees and their spatial arrangement within the hive are visualized in the secondary subplot. Both qualitative and quantitative analysis are supported by this graphical output, which allows users to easily observe agent behaviours, state transitions, and environmental changes throughout simulation steps.

Config Loading: The utility functions in config_loader.py allow for customizable simulation setup by loading slope maps and parameter values from CSV files. This facilitates rapid scenario testing, batch trials, and reproducibility by allowing users to modify simulation parameters and world layout without altering the main codebase.

UML Class Diagram



Showcase

Introduction

Three scenarios were set up to demonstrate the simulation's flexibility:

- **Scenario 1:** No nectar collected despite bee exploration.
- **Scenario 2:** Directed communication enabled successful nectar return.
- **Scenario 3:** High obstacles limited nectar collection.

How to Run Each Scenario

Scenario 1:

Command: *python3 beeworld.py -i*

Input:

- World size: 15x15
- Number of bees: 10
- Number of flowers: 15
- Number of obstacles: 3
- Simulation steps: 10
- Bee communication strategy: random

Scenario 2:

Command: *python3 beeworld.py -f map1.csv -p params1.csv*

Input:

- World size: 10x10
- Number of bees: 10
- Number of flowers: 10
- Number of obstacles: 3
- Simulation steps: 10
- Bee communication strategy: directed

Scenario 3:

Command: *python3 beeworld.py -i*

Input:

- World size: 10x10
- Number of bees: 5

- Number of flowers: 5
- Number of obstacles: 8
- Simulation steps: 15
- Bee communication strategy: random

Scenario 1

Output:

Throughout the simulation, bees spent the first 5 steps entirely in the hive or resting, with 10 bees consistently inactive. From step 6 onward, up to 10 bees began searching, with several (e.g., bee_0, bee_1, bee_2, bee_3, bee_5, bee_9) finding flowers and at least 1 bee finding an empty flower. Despite 15 flowers present, no bees collected nectar, and the hive nectar level remained at 0 for all 10 steps. No flower locations were shared, and flower nectar levels (Empty=3, Level1=3, Level2=2, Level3=7) stayed unchanged throughout.

Scenario 2

Output:

In Scenario 2, bees remained inactive or resting for the first five steps, with all 10 bees in the hive. From step 6, bees began searching and several found flowers or empty flowers. By step 8, 7 bees were collecting nectar, and by step 9, 5 bees were returning to the hive. At the end of 10 steps, hive nectar increased to 5, with 5 bees in the hive, 1 still collecting, and 5 searching. Flower nectar levels shifted, with empty flowers increasing from 3 to 5, indicating successful nectar collection and return.

Scenario 3

Output:

During Scenario 3, bees began by resting in the hive for the first five steps. Starting from step 6, bees transitioned to searching and gradually located flowers, with some collecting nectar in later steps. Obstacle density appeared to slow bee progress, as only a few bees managed to collect and return nectar. By the end of 15 steps, the hive nectar level reached just 1, with 2 bees returning and 3 still searching. Flower nectar levels showed a slight increase in empty flowers, reflecting limited but successful nectar collection.

Conclusion

The Bee World Simulation meets the assignment specification by modelling bee foraging with agent-based logic, supporting different strategies and world setups, and providing both terminal and graphical output. The modular code design and clear class structure facilitate easy extension and testing.

Future Work

- Add queen and drone bee roles with different behaviours.
- Implement pheromone trails or more advanced communication.
- Support for saving/loading simulation state.
- Add parameter sweep and automated batch testing.

References

BrianMburu. n.d. “GitHub - BrianMburu/Artificial-Bee-Colony-in-python: Artificial Bee Colony Algorithm Implementation in Python.” GitHub. https://github.com/BrianMburu/Artificial-Bee-Colony-in-python.

Engineero. n.d. “GitHub - Engineero/Honeybee: An Artificial Bee Colony Implementation in Python.” GitHub. https://github.com/Engineero/honeybee.

“Hive.” n.d. Hive. https://rwuilbercq.github.io/Hive/.